Assignment 5

I. Hosting & Deployment Strategy

Host Site

- **Target Server/Cloud:** The application is designed as a **standalone desktop application**.
- **Deployment Location:** * **Core Application:** Deployed locally on a **Linux-based operating system** (Ubuntu recommended).
    - **AI Component:** The LLM logic is hosted remotely and accessed via an **external LLM API**.

Deployment Strategy

1. **Server/Environment Configuration:** The local machine must have **Python** installed. The application utilizes the **Qt framework** to render the terminal UI.
2. **API Configuration:** Security keys for the LLM API must be configured as **environment variables** on the host machine to ensure they are not hardcoded.
3. **Initiating Communication:** The **CommandController** acts as the bridge. It captures input from the TerminalUI, sends it to the LLMService via an HTTPS API call, and receives the generated shell command for execution.

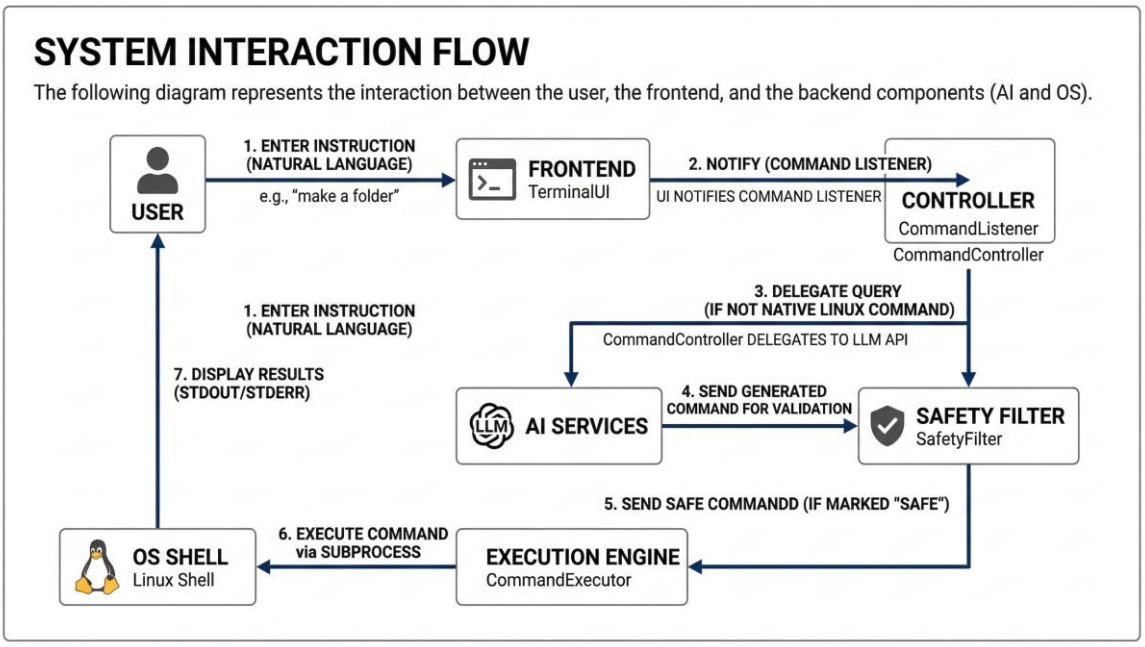
Security Measures

- **Command Safety Filter:** Every generated command is passed through a **Safety Module** that validates it against predefined safety rules.
- **Destructive Command Blocking:** The system specifically blocks commands involving system deletion, unauthorized access, or shutdowns.
- **Privilege Restriction:** All execution is restricted to **user-level privileges**, preventing the AI from performing root-level system changes.

II. Access and System Interaction

Users access the service through a **Terminal-Style Graphical User Interface (GUI)** that resembles a standard Linux terminal.

System Interaction Flow

The following diagram represents the interaction between the user, the frontend, and the backend components (AI and OS).

1. **User -> Frontend:** The user enters a natural language instruction (e.g., "make a folder") into the TerminalUI.
2. **Frontend -> Controller:** The UI notifies the CommandListener.
3. **Controller -> AI Services:** If the command is not a native Linux command, the CommandController delegates the query to the LLM API.
4. **AI Services -> Safety Filter:** The generated command is sent to the SafetyFilter for validation.
5. **Safety Filter -> Execution Engine:** If marked "Safe," the command is sent to the CommandExecutor.
6. **Execution Engine -> OS Shell:** The command is executed in the Linux shell via a subprocess.
7. **OS Shell -> User:** The execution results (stdout/stderr) are displayed back on the TerminalUI.